

Epistemic Memory: A Design Space for Belief-Aware AI Memory Systems

Dr. Laszlo Attila Vekony

Most production "AI memory" systems in 2024–2026 have been variations on retrieval — vector indexes, key-value stores, summarized rolling buffers — and have addressed primarily the question *what is similar to this query?* As AI systems take on more analytical and agentic work, a second question increasingly matters: *what does the system believe, with what evidence, and how does that change over time?* We name the class of memory systems concerned with the second question epistemic memory and argue that it constitutes a recognizable design space distinct from retrieval-augmented memory, off-the-shelf knowledge graphs, and probabilistic graphical models, while drawing useful primitives from each. We articulate a set of concerns that distinguish work in this space — differential trust, preserved disagreement, causal traceability, temporal honesty, edge-level provenance, and composability for multi-agent operation — and offer a design-space framework along ten dimensions of independent choice. We map several existing systems (Datomic, W3C PROV, Markov logic networks, RDF-star, vector stores with metadata, the agent-memory layers of LangGraph and AutoGen, and the MemEX library) as partial implementations occupying different regions of this space. We do not claim to define a closed category; rather, we offer vocabulary and dimensions intended to make trade-offs explicit, support comparison between implementations, and identify regions that remain underexplored.

epistemic memory | belief representation | AI memory | knowledge graph | provenance | design space

Introduction

"Memory" in an AI system can mean several different things, and conflating them has produced confusion that this paper attempts to clear. At least four distinguishable functions are commonly grouped under the term:

1. **Recall** — given a query, surface text or records that resemble or contain related material. The dominant production answer is vector similarity.
2. **State** — track the evolving content of an ongoing interaction (turn-to-turn buffers, summarized rolling context, scratchpads).
3. **Knowledge** — maintain a structured representation of entities, relationships, and facts that can be queried symbolically. Knowledge graphs and triple stores instantiate this.
4. **Belief** — represent what the system *holds to be true*, with evidence, provenance, conflict, and temporal validity, in a way that supports defensible reasoning and audit.

The first three are well-served by mature technology: vector indexes, conversation buffers, RDF and property-graph databases. The fourth — belief — has been treated less systematically. It is sometimes addressed as a side effect of one of the other three (a knowledge graph "happens to" record sources; a vector index "happens to" return only recent items), and sometimes addressed by sophisticated but operationally heavy probabilistic frameworks (Bayesian networks, Markov logic networks). Neither approach has produced a layer that AI engineers can adopt without either substantial implementation work per project or substantial mathematical infrastructure that does not match the texture of the domains.

We argue that **belief** deserves its own substrate, and that practical work in this direction is already happening across multiple projects under different names and with different design choices. We propose the term **epistemic memory** for this class of systems, and we offer a vocabulary and design-space framework intended to make the design choices explicit and to support comparison between approaches.

This paper is a position paper, not a benchmark paper. We do not evaluate systems empirically; we map a design space. The contribution is conceptual: a name for the category, an articulation of the concerns it addresses, a set of design dimensions along which implementations vary, and a partial plot of existing systems as points in that space. We close with open problems where, in our view, the literature and the practitioner community would benefit from coordinated progress.

2. What Epistemic Memory Is — and Is Not

We define epistemic memory operationally rather than formally. Epistemic memory is the class of memory systems whose primary purpose is to represent, maintain, and answer questions about *what an AI system believes*, where "what is believed" includes:

- **What** is asserted (the content of a belief)
- **Who** asserts it (the source or chain of sources)
- **How much** it should be trusted (one or more axes of confidence/authority)
- **Why** it is held (the supporting evidence, recursively)
- **What conflicts with it** (contradicting beliefs, with their own provenance)
- **When it was held, and whether it still is** (temporal validity, supersession history)

Two negative definitions help locate the category:

Epistemic memory is not retrieval. Retrieval answers *what is similar to this query?* Epistemic memory answers *what does the system believe about this topic, and on what grounds?* The two are complementary: most production systems will use both. A vector index is excellent recall infrastructure; it does not, by itself, distinguish a satirical tweet from an audited filing or surface a contradicting source alongside a primary one. An epistemic memory layer can sit above retrieval, consuming candidate items from the index and operating on them with belief semantics.

Epistemic memory is not a probabilistic graphical model. Bayesian networks, Markov logic networks, and probabilistic logic programming represent belief in mathematically principled ways using conditional probabilities and explicit factor graphs. They are powerful and clean for problems whose schema is fixed and whose probabilities are calibratable. They are, in our experience, operationally heavy for the open-vocabulary, novel-entity, "I am 80% sure that the analyst is 60% sure" texture of analytical AI work, where trust about the world and trust about the speaker need to be modeled separately and tracked over time. We see probabilistic graphical models and epistemic memory layers as complementary rather than competing — an epistemic memory layer can hold a probabilistic engine's output as one kind of high-authority derived item among others.

A useful one-line characterization: epistemic memory is to belief what vector search is to recall and what knowledge graphs are to facts. It is its own thing, and the category is open.

3. Concerns That Distinguish Epistemic Memory Work

We articulate six concerns that, in our observation, drive most of the design choices in systems we would describe as epistemic memory. These are not requirements; they are areas of attention that distinguish belief-aware design from retrieval-only or fact-only design. Different systems weight them differently, and that is appropriate — the category is open, and the concerns admit different reasonable answers.

3.1 Differential trust. Not all sources are equal, and the difference is rarely binary. A useful epistemic memory can rank claims by who said them in a way that is independent of how recent or how popular they are. Treating an audited regulatory filing and an anonymous social-media post as semantic peers — as a vector index will, by default — is for most analytical purposes a defect.

The natural representational move is one or more numerical axes per item, expressing trust at different granularities (system-level trust, author-internal confidence, operational salience). Systems vary in whether they expose one such axis, several, or none.

3.2 Preserved disagreement. When two credible sources conflict, the analytical task usually requires *seeing both*. Silent

resolution — picking one and discarding the other — produces confidently wrong answers in any domain where disagreement is itself an informative signal. Legal practice contains this property explicitly (overruled cases remain queryable; both sides of a circuit split are briefable). Geopolitical analysis contains it operationally (an outlet pushing back hard on a fact may be evidence that the fact is true). Financial analysis contains it implicitly (bull and bear theses on the same name should not be averaged into a flat "consensus" view).

The representational move is contradiction-as-data: a first-class structural relationship between conflicting items, queryable and surfaceable rather than overwritten. Resolution, when it happens, is itself a recorded action with its own provenance.

3.3 Causal traceability. Most analytical conclusions need to be answerable to *what evidence justifies this?* — recursively, to root observations. This is sometimes a regulatory requirement (credit ratings, compliance opinions, medical recommendations), often a professional one (legal advice, equity research notes), and almost always useful in debugging the system itself.

The representational move is provenance as a graph rather than a string. Each derived belief carries explicit links to the items it was derived from; traversal upward yields the full justification chain. The granularity of this provenance — whether it is per-document, per-section, per-claim, per-sentence — is itself a design choice (§4.5).

3.4 Temporal honesty. Claims age. Distinguishing *true now* from *true when first asserted*, without rewriting history, matters in markets ("as of this date"), in law ("controlling authority on this date"), and in geopolitics (post-mortem on prior assessments). It also matters in agent self-evaluation: the question "was my assessment six months ago consistent with what I knew at the time?" is a fundamentally different question from "is my current assessment correct?"

The representational move is some combination of: append-only history, point-in-time query semantics, and decay/staleness models that operate at retrieval time without rewriting stored content. Systems vary in which of these they support and whether decay is fixed or query-configurable.

3.5 Edge-level provenance. A frequently underweighted concern: the *relationships* between items are themselves claims with sources, confidence, and possible conflict. The assertion that "case A overrules case B" is a claim by some authority; the assertion that "filing X supports thesis Y" is a claim by some analyst at some level of confidence. Encoding these only as edges of a typed graph, with no provenance of their own, loses the information that an audit usually needs.

The representational move is to treat edges as first-class objects with the same metadata structure as nodes — author, source kind, authority, possibly contradiction with other edges. RDF-star is a partial step in this direction; most property-graph implementations are not.

3.6 Composability for multi-agent operation. This concern is more recent than the others and reflects the move toward agentic AI. As multiple agents — and sometimes multiple humans alongside them — operate on a shared belief state, the memory layer needs to support patterns that flat systems cannot:

- **Subset operation** — agents specializing on portions of the graph, not the whole.
- **Sandboxing** — sub-agents reasoning on copies of the graph without contaminating consensus.
- **Conflict-aware merging** — divergent edits resolved as data, not as last-writer-wins.
- **Cross-session continuity** — beliefs persist across user conversations without an explicit "long-term memory" / "conversation memory" split.
- **Bounded autonomous reasoning** — agents thinking between user prompts, prioritized by belief uncertainty.

The representational moves include exportable slices, append-only with conflict reporting, and a coordinated representation of goals and tasks alongside beliefs.

These six concerns are not a checklist that any "epistemic memory" implementation must satisfy. They are a vocabulary for talking about which problems a given system addresses, which it punts to the application layer, and which it ignores. Most existing systems address some subset; few address all six well; the open ones in the literature largely concern §3.4 (temporal honesty under uncertainty) and §3.6 (multi-agent composability).

4. The Design Space

For each concern in §3, the question *how* it is addressed admits multiple reasonable answers. We articulate ten design dimensions along which implementations vary independently. A specific epistemic memory system is a point in this space.

4.1 Storage model. How is the belief state represented?

- **Property graph** — typed nodes and edges with arbitrary metadata
- **RDF / triple store** — subject-predicate-object triples, possibly with annotation (RDF-star)
- **Append-only event log** — state is a fold over a sequence of commands
- **Probabilistic factor graph** — nodes are random variables, edges are factors
- **Hybrid** — typically a graph layered on an event log, or a graph with a complementary index

Trade-off: graph models give natural traversal but mutate in place; event logs give time travel and replay but require materialization for query.

4.2 Trust model. How is the confidence in a belief represented?

- **None** — beliefs are flat (a vector store with metadata that the application interprets)
- **Single binary** — true / not-true (classical KGs)
- **Single scalar** — a confidence float (many ad-hoc systems, some probabilistic logic programs)
- **Multi-axis scalar** — separate axes for system trust, author confidence, operational salience, etc.
- **Probability distribution** — full Bayesian posterior

Trade-off: more axes capture more nuance at the cost of authoring burden and interpretability.

4.3 Conflict policy. What happens when two beliefs disagree?

- **Silent resolve** — last write wins, or some merge function
- **Surface** — both are returned, marked as conflicting
- **Resolve with history** — explicit resolution event records winner, loser, and reason; both remain queryable
- **Probabilistic merge** — beliefs are combined via a posterior update rule

Trade-off: silent resolution is simpler but loses information; surface and resolve-with-history preserve information at the cost of a richer query surface.

4.4 Temporal model. How is time and history represented?

- **Snapshot** — only current state is kept
- **Append-only with point-in-time** — full history queryable, no branching
- **Branching with merge** — multiple worldlines from the same baseline, mergeable
- **Branching without merge** — multiple worldlines that never reconverge
- **Bitemporal** — separate axes for "valid time" (when a fact is true in the world) and "transaction time" (when the system learned it)

Trade-off: branching with merge is the most expressive but also the most demanding to implement correctly; bitemporal is the most rigorous but adds query complexity.

4.5 Provenance granularity. At what granularity are sources tracked?

- **None**
- **Per-document** — "this came from filing X"
- **Per-section** — "this came from the MD&A of filing X"
- **Per-claim** — "this came from sentence Y of section Z of filing X"
- **Chain-of-derivations** — full graph of how each belief was derived from others

Trade-off: finer granularity supports better audit and recovery from source revision but requires more sophisticated ingestion and storage.

4.6 Identity resolution. How are the same entity's different surface forms reconciled?

- **Name match** — string equality (or simple normalization)
- **Explicit aliasing** — the system records "X and Y are the same"
- **Embedding-based** — entities are clustered by similarity
- **Heterogeneous / federated** — IDs are external (e.g., Wikidata Q-IDs)

Trade-off: explicit aliasing is auditable; embedding-based scales but introduces probabilistic identity.

4.7 Inference engine. What kinds of inference does the system natively perform?

- **None** — application code derives all conclusions
- **Built-in deterministic rules** — Datalog, structural inheritance, rule-based derivation with provenance
- **Built-in probabilistic** — MLN inference, ProbLog inference, learned graph-neural-network derivation
- **Hybrid** — deterministic by default, probabilistic for specific layers
- **External** — integration points for arbitrary external reasoners

Trade-off: built-in inference simplifies the application but couples it to the engine's semantics; external integration is flexible but pushes more work to the application.

4.8 Persistence and scale. Where does the data live?

- **In-memory only** — process-local, no durable state
- **Single-node durable** — local database
- **Distributed** — sharded across nodes
- **Federated** — multiple independent stores with cross-store queries

Trade-off: in-memory-only is fastest and simplest but does not scale; federation is most flexible but introduces consistency challenges.

4.9 Query model. How are beliefs retrieved?

- **Structural traversal** — graph queries (Cypher, SPARQL, Gremlin)
- **Ranked retrieval** — beliefs returned ordered by score with budget cutoff
- **Both** — composable structural filter + ranked output
- **Probabilistic query** — answers come back as distributions or marginal probabilities

Trade-off: traversal is precise but does not handle "give me the top N most relevant"; ranked retrieval is the natural mode for AI-agent context-window packing.

4.10 Multi-agent coordination. How do multiple agents share state?

- **None** — single agent, single graph
- **Shared mutable graph** — agents read/write the same store, application-level locking
- **Slice export-import** — agents work on copies, merge back with conflict reporting
- **CRDT-like** — convergent data types ensure eventual consistency without coordination
- **Federated** — agents have separate stores, exchange messages

Trade-off: slice export-import is conceptually simple and fits sub-agent / crew patterns; CRDTs are stronger guarantees but more constrained representation.

—

These ten dimensions are independent in the sense that a system's choice on one does not strictly determine its choice on another. They are not orthogonal in the engineering sense — some combinations are natural (event log + branching + slice export-import; probabilistic engine + probability distribution trust; in-memory + structural traversal), and others are awkward — but no single dimension dictates the rest. A specific implementation is a vector in this space.

5. Existing Systems as Partial Implementations

We plot several existing systems as points in the design space described above. The mapping is necessarily compressed; readers familiar with each system will note simplifications, and we welcome corrections. The purpose is to illustrate that the design space is well-populated and that no system currently occupies all interesting corners — which is, in our view, evidence that the category is open and worth coordinated attention rather than a defect of any individual system.

A few observations from this map that, in our view, suggest where coordinated work would be valuable.

The retrieval columns and the belief columns are not the same systems. Pinecone-style vector stores and Weaviate handle ranked retrieval beautifully but provide no native trust, conflict, or provenance. Datomic and the RDF family handle structural and temporal queries beautifully but provide little or no native trust scoring. Practitioners routinely glue these together application-side; the gluing is where epistemic-memory bugs hide.

Probabilistic systems address belief but at a different operating cost. MLNs and ProbLog are mathematically clean for fixed schemas but operationally heavy for the open-vocabulary, novel-entity texture of analytical AI. They also typically lack agent-pattern composability: there is no notion of "export a slice for a sub-agent."

Agent-memory layers in 2024–2026 frameworks are weak on belief. LangGraph state and AutoGen-style memory are turn-to-turn focused, with little to no native trust, contradiction, provenance, or temporal honesty. They are excellent state machines; they are not belief stores.

Edge-level provenance is rare. Only RDF-star and W3C PROV directly support claims-about-claims as a first-class structural feature. Most graph systems force this into node metadata or out-of-band annotations.

Multi-agent composability is rare. Slice export-import as a first-class operation, with append-only conflict-aware merging, appears explicitly only in MemEX among the systems surveyed. CRDT-based memory for AI agents is, to our knowledge, an open problem.

A separate consequence of this map is that *no system "wins."* Every implementation makes trade-offs. The point of articulating the space is to make those trade-offs visible and to support deliberate selection rather than accidental adoption.

6. The Agent-Pattern Test Case

We single out one consequence of the design choices because, in our view, it is where the value of epistemic memory most visibly compounds: support for **agent patterns** that go beyond single-prompt reasoning.

A useful epistemic memory layer for AI agents tends, in practice, to enable four patterns that retrieval-only or knowledge-graph-only systems struggle to support natively:

- **Crews** — small teams of agents specializing on subsets of the graph (typically by `scope` or by some other partition key), coordinating through the graph rather than through point-to-point messages
- **Swarms / sub-agents** — transient fan-out from a shared baseline, with each sub-agent operating on a copy of the slice and merging back with conflict reporting
- **Cross-session memory** — beliefs that persist across user conversations and across days, queryable as a unified graph rather than split between "long-term memory" and "conversation memory"
- **Background thinking** — bounded autonomous reasoning between user prompts, prioritized by belief uncertainty (high importance, low authority) and anchored to existing intents, surfacing results to the user when something crosses an attention threshold

These patterns are not features of any specific system; they are *constructible* from the right combination of design choices. Specifically, they tend to require: append-only history (for cross-session continuity and audit), slice export-import (for crews and swarms), multi-axis trust (for prioritizing what is worth thinking about), edge-level provenance (for tracing autonomous-thinking outputs), and a coordinated representation of intents and tasks alongside beliefs (for goal-anchored background work).

This combination is the corner of the design space that, in our experience, currently shows the largest gap between what AI engineers are building and what existing memory infrastructure natively supports. Filling the gap is, we suggest, the most operationally consequential open problem in the space.

7. Composition with the Wider Agent Stack

Epistemic memory is one component in a deployed AI agent, not a standalone system. We have not seen working systems that treated belief substrate, retrieval, working-memory windowing, and identity/policy as a single layer. We deliberately do not prescribe an exact configuration or hierarchy for how these components compose: the right composition depends heavily on workflow, industry, and deployment priorities, and configurations that work well in one context (regulated, audit-heavy advisory work) will not be the right fit for another (a fast-iterating consumer assistant, a real-time trading copilot, a research agent operating on archival data). What we describe here is the set of components an epistemic memory layer typically composes with, not a recommended layering of them.

Sliding-window working context. The LLM's immediate context window — the active conversation buffer plus whatever has been loaded for the current turn — is volatile working memory, distinct from a persistent belief substrate. In observed deployments, some portion of the substrate is loaded into the window per turn, while fresh utterances accumulate in a rolling buffer. The exact ratio of pre-loaded belief context

System	Storage	Trust	Conflict	Time	Provenance	Inference	Persistence	Query	Multi-agent
Vector + metadata (Pinecone, Weaviate)	sparse vector + KV	none / single scalar	silent	snapshot	per-document	none	distributed	drank	none
Datomic	graph + event log	none	silent	append-only + bitemporal	per-fact	datalog rules	distributed	structural + temporal	shared mutable
Neo4j (+ provenance plugins)	property graph	none / per-property	silent	snapshot	per-document (plugin)	cypher or pattern matching	single-node or distributed	structural	shared mutable
RDF-star	triple store + annotation	per-edge meta-data	silent	snapshot	per-edge	SPARQL	single-node or distributed	structural	shared mutable
W3C PROV (ontology)	RDF	metadata-only	n/a	n/a	per-fact	n/a	varies	varies	n/a
Markov Logic Networks	factor graph	probabilistic distribution	probabilistic merge	snapshot	implicit (factors)	probabilistic inference	single-node	probabilistic query	none
ProbLog	probabilistic Datalog	probabilistic distribution	probabilistic merge	snapshot	rule-derivation	probabilistic inference	single-node	probabilistic query	none
LangGraph state	typed dict / pydantic	none	silent (overwrite)	snapshot per node	none	none	in-memory	direct access	shared mutable within graph
AutoGen memory	text buffer + summary	none	silent	rolling window	none	none	in-memory or external	text retrieval	none
MemEX	property graph + event log	three-axis scalar	surface or resolve-with-history	append-only + branching via slice export	per-claim, chain-of-derivations	none (deterministic via app)	in-memory + ranked	structural	slice export-import

to live conversation buffer, the retrieval policy, and the refresh cadence are workflow choices.

Personality and policy via system prompts. Agent identity, role configuration, and behavioral policy typically live in the system prompt, separate from the belief substrate. Keeping these layers separate produces, in our observation, more maintainable systems than fusing them. Placement of fuzzy content (durable user preferences, learned style adjustments) is deployment-specific, with reasonable arguments on either side.

Retrieval infrastructure. As argued in §2, retrieval and belief are complementary, and both are typically present. The directionality of the relationship is workflow-dependent: search results may enter the substrate as candidate observations to be assigned provenance and reasoned about; the substrate may provide query terms or filter constraints for downstream search; both flows often coexist. The choice often follows from whether the deployment is primarily belief-driven (retrieval as a tool) or primarily retrieval-driven (belief as an annotation layer).

External and unexplored sources. A useful agent rarely has a complete information environment at start. Web search,

official APIs, internal company systems, the user’s own files, third-party services — these are sources reached through tools rather than necessarily ingested wholesale. Task-driven exploration is one composition pattern: a hypothesis with low authority and high importance triggers a task whose action is to consult an external source, with the result entering the substrate as a new observation with provenance. Whether to bulk-ingest a source upfront, consult it lazily, or do some hybrid is a deployment-specific decision driven by latency, cost, and data-volatility constraints rather than by the substrate.

The general point: an epistemic memory layer is one of several components in an agent architecture, not a replacement for any of them. We have seen many different working configurations of how these components compose, and we do not recommend a canonical shape. We do recommend that authors of new implementations make their composition story explicit *for the workflows and industries they target*, while recognizing that a different workflow may justify a different composition.

This recommendation has a corollary for the design space discussed in §4: the multi-agent coordination dimension (§4.10) is, in deployed practice, often the dimension that interacts most strongly with what the agent looks like as a whole —

which retrieval tools are routed through which agent, which sources are reached by which task, how sub-agents see partial views of the belief substrate. We expect this dimension to receive disproportionate attention as deployed agentic systems mature, and we expect deployment-specific configuration patterns rather than a single converging shape.

8. Open Problems

We list problems that, in our view, are not adequately addressed by any current system in the surveyed landscape, and where coordinated progress would be valuable. These are not all problems for the substrate layer; some sit at the application layer and are listed because they substantially affect what a useful substrate needs to support.

8.1 Authority calibration. A trust score is only as good as its calibration to ground truth. Hand-curated source registries dominate current practice but scale poorly. Learned calibration — using past prediction outcomes, agreement-with-future-evidence rates, or external benchmarks — is conceptually clean but requires labeled outcome data that is rarely available at the granularity of individual sources. We are not aware of a principled, domain-general framework here.

8.2 Contradiction detection. Epistemic memory systems make contradictions easy to *represent*. Detecting them is a separate, hard problem, and is highly domain-specific (legal-clause contradiction is a different problem from event-narrative contradiction is a different problem from forecast contradiction). The substrate layer can consume detection results; the detection itself is a research area in its own right and is, we believe, currently under-served by the AI-agent community.

8.3 Decay and staleness parameter learning. Real-world claim half-lives vary by orders of magnitude across domains — a tactical OSINT report decays in hours, a constitutional precedent in centuries. Choosing decay parameters per source kind, per claim type, or per scope is currently a hand-tuning problem. Learning these parameters from the rate at which claims are revised, retracted, or superseded is a natural direction; we are not aware of mature work in this area.

8.4 Probabilistic-symbolic integration. Epistemic memory systems with multi-axis scalar trust (rather than full probability distributions) make heuristic combination decisions when ranking — and those heuristics are not principled posteriors. For applications where the math matters (portfolio construction, drug-interaction risk, formal compliance), the substrate should compose with probabilistic engines rather than replace them. Practical integration patterns — where in the stack the symbolic and probabilistic layers meet, and how trust scores translate to and from explicit probabilities — are an open design problem.

8.5 Adversarial robustness. In domains where adversaries actively shape the source landscape (geopolitics, financial markets, regulated industries), the substrate layer needs

not just to *represent* trust but to be *robust* against coordinated low-authority spam, source-impersonation attacks, and provenance-graph poisoning. Most current epistemic memory work is silent on this; the security-research community has not, in our reading, focused on it.

8.6 Cross-system interoperability. W3C PROV is the closest existing standard for portable provenance. There is no equivalent standard for the multi-axis trust, contradiction-relationship, or branching-history primitives that many epistemic memory systems share. As multiple implementations occupy the design space, an interchange format — analogous to OpenAPI for HTTP services or Apache Iceberg for tabular data — would let beliefs move between systems without lossy translation. We see this as a coordination problem more than a research problem, but the coordination has not yet happened.

8.7 Identity resolution at scale. Across millions of items, deciding when two surface forms refer to the same entity is currently solved either by exact match (brittle), explicit aliasing (high authoring cost), or embedding similarity (probabilistic, hard to audit). Identity resolution that is scalable, auditable, and correctable is, we believe, an open research problem that deserves more attention from the epistemic-memory community than it currently gets.

8.8 Multi-agent CRDT-like memory. Slice export-import with append-only merge is one approach to multi-agent shared belief. CRDT-based approaches — where the conflict resolution is built into the data type rather than handled at merge time — are theoretically appealing but require careful design to fit belief semantics (an authority score with last-writer-wins CRDT semantics may not be what you want). We are not aware of mature CRDT-based epistemic memory implementations.

References

This is a position paper; the bibliography focuses on works that establish the historical and conceptual context for the category, not on a comprehensive survey. We have elected to keep the reference list short and to cite only works we have used substantively in the argument.

1. Pearl, J. (1988). *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann. (For the probabilistic graphical model frame discussed in §2 and §4.7.)
2. Richardson, M., & Domingos, P. (2006). Markov Logic Networks. *Machine Learning*, 62(1–2), 107–136. (Cited in §5.)
3. De Raedt, L., Kimmig, A., & Toivonen, H. (2007). ProLog: A Probabilistic Prolog and Its Application in Link Discovery. *IJCAI*. (Cited in §5.)
4. Hartig, O. (2017). Foundations of RDF-star and SPARQL-star: An Alternative Approach to Statement-Level Metadata in RDF. *AMW*. (Background for RDF-star, §5.)

5. Lebo, T., Sahoo, S., & McGuinness, D. (eds.) (2013). PROV-O: The PROV Ontology. *W3C Recommendation*. (Background for W3C PROV, §5.)
6. Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*. (For the RAG baseline contrasted in §1.)
7. Hickey, R. (2012). The Datomic Database. *Cognitect technical documentation*. (Background for the event-sourced bitemporal model, §5.)
8. Shapiro, M., Preguiça, N., Baquero, C., & Zawirski, M. (2011). Conflict-Free Replicated Data Types. *SSS*. (For the CRDT background and the open problem in §8.8.)
9. MemEX library (2026). @ai2070/memex on npm. Working draft whitepaper: *MemEX: An Epistemic Reasoning Graph for AI Agents*. (Companion to this paper; contains worked examples and full design rationale for one system in the space.)

Disclosure

The author of this paper is the developer of the MemEX library, which is plotted alongside other systems in §5 and discussed at greater length in a companion whitepaper. We have been deliberate about positioning MemEX as one practical approach among others rather than as a reference implementation, and about not defining the category's design space to fit MemEX's feature list. Readers should weigh the framework offered here with this conflict of interest in view. We welcome corrections, particularly to the survey table in §5, where compression has likely produced characterizations that experts on individual systems will find imprecise.

This paper has not been peer-reviewed. It is offered as a position paper / working draft for public release, intended as a starting point for conversation rather than a finished contribution.

Discussion

The framework offered here is deliberately non-prescriptive. We have not argued that any specific system is "correct" or that any specific design choice is "best." We have argued that:

1. There is a recognizable class of memory systems concerned with belief rather than retrieval.
2. The class admits a meaningful set of concerns, distinct from those of pure retrieval or pure probabilistic inference.
3. Within the class, design choices vary along ten reasonably independent dimensions, and a specific system is a point in that space.
4. Existing systems occupy disparate, partial regions of the space; no system covers all interesting corners.

5. The most operationally consequential gaps appear to be in multi-agent composability, temporal honesty under uncertainty, and the integration of probabilistic and symbolic reasoning.

The framework is intended to be useful in three ways. For practitioners building AI products, it offers a vocabulary for evaluating candidate memory layers and for explaining trade-offs to stakeholders. For researchers, it offers a coordinate system for situating new contributions and for comparing across implementations. For the community as a whole, we hope it can support the gradual emergence of standards and interchange formats that let beliefs move between systems without lossy translation.

We are an interested party. One of the systems plotted in §5 — MemEX — is developed by the author. We have made the cross-references to it explicit and have been careful to position it as one practical approach occupying one corner of the space, not as a reference implementation. The whitepaper that accompanies this paper [REF] develops MemEX's design choices in greater depth, with worked examples in financial, legal, and geopolitical analysis. Readers interested in a concrete reference for one set of choices in this space can begin there. Readers interested in alternative or competing implementations are, we hope, the primary audience for the framework offered here.

Conclusions

The retrieval-vs-belief distinction has been doing real work in AI memory systems for several years without being named. Naming it is the first contribution of this paper. The second is articulating the concerns that distinguish belief-aware design from retrieval-only design. The third is the design-space framework along ten dimensions of independent choice. The fourth is a partial map of existing systems as points in that space.

We do not claim the category is closed, the concerns are exhaustive, or the dimensions are the only ones that matter. We claim that the category is real, that the concerns are worth considering, and that the dimensions usefully expose the trade-offs that current implementations make implicitly. We invite further implementations occupying corners of the space we have not built, and we invite revisions to the framework itself — what we have offered is vocabulary, not doctrine.

ABOUT THIS MANUSCRIPT

This manuscript was prepared using [Rxiv-Maker v1.22.0 \(1\)](#).

EXTENDED AUTHOR INFORMATION

Bibliography

1. Bruno M. Saraiva, António D. Brito, Guillaume Jaquemet, and Ricardo Henriques. Rxiv-maker: an automated template engine for streamlined scientific publications, 2025. doi:[10.48550/arXiv.2508.00836](https://doi.org/10.48550/arXiv.2508.00836).